AgentLoopGate: Evidence-Governed Evolution and Fail-Closed Selection for Agent Harnesses

JiaoyangLi
jiaoyanglifly@gmail.com
https://github.com/jiaoyangli-shadow7day/AgentLoopGate

Draft – August 2026

Abstract

Agent harnesses can now modify their own prompts, tools, routing rules, and control flow. The difficult question is no longer only how to generate a change, but how to decide whether the evidence is strong enough to release it. We present AgentLoopGate, a runtime-independent governance layer for evaluating harness changes before release. The system preserves the host runtime's native traces and telemetry, while adding immutable experiment identities, complete attempt and cost accounting, independent baseline-paired evaluation, and a fail-closed decision gate that stops when evidence is incomplete or no candidate qualifies. Candidate generators remain interchangeable, and promotion is never automatic. We evaluate the design in two ways. First, we trace how failures observed across P0–R14 led to the current evidence contracts. Second, we conduct a frozen Banking study with DeepSeek Harness and an external updater. The study completed all 125 authorized pre-release evaluations and compared a baseline with three generated candidates on the same 15-task Selection set. Two candidates matched the baseline's aggregate score and one scored lower, but paired analysis showed that every candidate lost previously successful behavior; all candidates also exceeded the frozen latency limit. AgentLoopGate therefore returned Hold and started no Release evaluation. Exact known model cost was USD 3.9086647880. Plugin fixtures also confirmed coexistence with native JSONL/SQLite persistence and OpenTelemetry. No positive self-evolution effect is claimed. Instead, the study shows how evidence governance can expose capability trade-offs and make abstention a reproducible, auditable outcome.

1 Introduction

An agent harness turns a language model into a working system: it supplies tools, maintains state, recovers sessions, and decides how model outputs become actions. Because the harness controls so much of an agent's behavior, changing the harness can improve the agent without changing model weights. Recent work therefore explores reflection [10, 7], learned prompt and pipeline parameters [4], function-level optimization [13], and automated search over agent programs [3].

Generating a plausible change, however, is only the first half of the problem. The second half is deciding whether that change should be released. Consider a candidate that solves two tasks the baseline misses but breaks two tasks the baseline already solves. Its aggregate score is unchanged, although its behavior is not. A retry can similarly hide an earlier failure, a missing trace can silently remove a task from the denominator, and an optimizer can appear successful if it is allowed to evaluate its own proposals. These are not merely statistical inconveniences. They are failures in the chain of evidence between an execution and a release decision.

AgentLoopGate addresses this decision problem. It sits beside an existing agent runtime and turns native execution records into evidence that can be checked before a candidate advances. The runtime continues to own sessions, traces, and telemetry. AgentLoopGate freezes the experiment definition, records every attempt and its cost, limits what the updater can see and change, compares each candidate with the baseline on the same tasks, and applies an ordered release gate. The gate may recommend shipping, reject invalid evidence, or simply hold when the study does not support a decision. A human remains the only promotion authority.

We study the system from two complementary perspectives. P0–R14 document how successive execution and evaluation failures shaped the design. These rounds are development history, not a pooled measure of effectiveness. R15 is a separately frozen Banking validation that exercises the resulting policy end to end. Its central result is deliberately negative: generated candidates changed behavior, but none preserved the baseline's successful behavior while improving it. The system detected the trade-off and stopped before Release.

This paper makes four contributions:

1. an evidence model that links native traces, normalized runs, candidates, costs, and decisions without replacing host persistence or telemetry;
2. a fail-closed protocol that separates candidate diagnosis from independent Selection and conditional Re-

lease, while retaining every attempt and its cost;
3. a DeepSeek Harness plugin that observes public session events while preserving native JSONL/SQLite persistence and OpenTelemetry; and
4. a sealed Banking study in which paired evaluation reveals regressions hidden by aggregate-score ties and correctly leads to abstention.

The empirical claim is intentionally narrow. AgentLoopGate detected unsupported evolution and stopped it before Release. We do not claim that the updater improved the agent, that the Banking result generalizes to other domains, or that fixture latency represents production load. The value demonstrated here is the ability to reach, explain, and reproduce a well-supported decision not to proceed.

2 From Candidate Generation to Release Decisions

2.1 The decision problem

Let A_0 denote the frozen baseline harness. An external updater studies a bounded set of failures and proposes candidates $C_1, \ldots, C_m$. The updater may rank its proposals, but that ranking is only a hypothesis: the updater has seen diagnostic evidence and may use an objective that differs from the release policy. A separate Selection stage must therefore judge every candidate on evidence the updater did not control.

A *candidate* is an immutable snapshot of the permitted harness assets, such as instructions, routing rules, or tool-facing configuration. It is not a new model and does not modify model weights. We call a task *stably successful* when it succeeds in every trial registered for that stage. R15 used one trial per Selection task, so stable success there is simply success on that task; a later Release stage would have required three successful trials. One *position* means one execution of a particular harness variant on one task in one registered trial.

For each task t in a frozen Selection set, let $y_t(A) \in \{0, 1\}$ indicate whether harness A succeeds stably. Choosing the largest value of $\sum_t y_t(A)$ is insufficient. It ignores which tasks changed and whether the comparison includes every registered task. Our release question is stricter: did a candidate add stable capability without removing protected baseline capability, and is that conclusion supported by complete, traceable evidence?

2.2 Design principles

Three principles follow from this question. First, proposal and judgment must remain separate. Diagnosis can guide an updater, but Selection and Release data must remain hidden until their authorized stages. Second, each conclusion must be reproducible from immutable identities for the objective, task split, evaluator, harness snapshot, runtime, and pricing. Retries, timeouts, missing cost, and infrastructure failures remain part of the record rather than being discarded as noise. Third, abstention is a valid outcome. If evidence is incomplete or no candidate clears the policy, the system should stop cleanly instead of forcing a winner.

These principles define a narrow trust boundary. The trusted kernel contains schemas, canonical serialization, content hashes, split enforcement, the evaluator interface, the candidate registry, the attempt and cost ledgers, and the final gate. Updater workspaces and mutable harness assets are untrusted. Raw model text, user simulation, and external telemetry may contribute evidence, but none can declare its own success.

2.3 Scope

AgentLoopGate governs evaluation and release recommendations; it does not train model weights, certify general agent safety, replace host logs, provide a trace viewer, or deploy a selected candidate. Banking is one reference validation environment, not the product boundary of the system.

2.4 A concrete pass through the pipeline

The stage names used throughout the paper describe a simple progression. *Update-Source* runs the baseline on diagnostic tasks and turns its failures into the only task evidence visible to the updater. The updater uses that bundle to construct one or more candidates. *Update-Check* runs a small, separate screening set so malformed or clearly harmful candidates can be removed cheaply. *Selection* then runs the baseline and every survivor on the same hidden tasks and may either choose one candidate or abstain.

Only a selected candidate can enter *Release*. Release-ID repeats the comparison on held-out in-distribution tasks, Release-OOD tests transfer to held-out workflow families, and Replay revisits preregistered earlier cases to detect forgetting. These stages use separate evidence and separate execution authority. Figure 1 shows where the records and decisions are created along this path.

3 System Design

Figure 1 follows one candidate from execution to a release decision. Each stage has a distinct responsibility. The host records what happened; AgentLoopGate turns that record into verifiable evidence; an updater proposes a bounded change; independent evaluation compares the change with the baseline; and the gate decides whether evaluation may advance. Keeping these responsibilities separate prevents a convenient trace, score, or proposal from silently becoming a release claim.

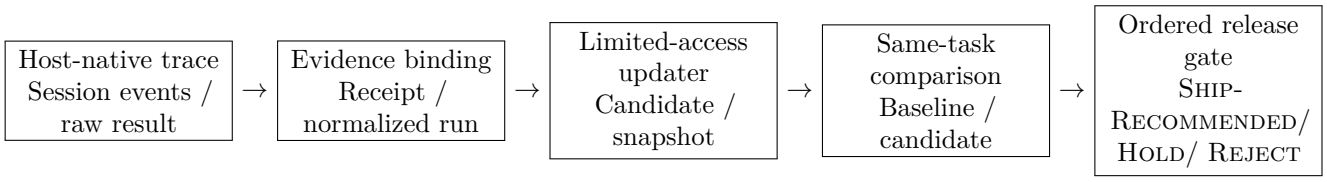

Figure 1: AGENTLOOPGATE separates runtime facts, governance evidence, candidate generation, and release decisions. The host trace remains the execution source of truth; content-addressed governance artifacts refer back to it.

3.1 From a native trace to decision evidence

The evidence model preserves both the original runtime record and the structured facts needed for comparison. The host runtime first stores its native append-only session log or benchmark result. AGENTLOOPGATE then adds a *SourceTraceRef*, which points back to those bytes, and a redacted receipt that proves which source was used. A normalized run extracts comparable facts such as tool use, final state, latency, token usage, and cost. Study-level artifacts finally combine those runs into failure bundles, candidate manifests, batch summaries, statistics, lineage, and decisions. Canonical JSON or YAML and SHA-256 digests bind each step to the one before it.

This layered design answers two different questions without confusing them. The native log answers, "What did the runtime record?" The normalized and study layers answer, "Is the evidence complete and comparable enough to support this decision?" A formal claim must be traceable through both.

Telemetry remains useful but is not treated as evaluation truth. OpenTelemetry, for example, is a vendor-neutral framework for exporting traces, metrics, and logs [8]; an exporter may be sampled, reconfigured, or temporarily absent. AGENTLOOPGATE can use such telemetry for diagnosis while basing decision integrity on the host event stream and authenticated receipts.

3.2 Frozen identities and immutable lifecycle

Before formal execution begins, the system freezes the objective, the isolated stage-specific task pools and replay set, evaluator, pricing, runtime versions, protocol, study plan, source revision, and baseline snapshot. These identities prevent a result from being quietly paired with a later evaluator, price table, or harness revision.

Execution then follows an append-only lifecycle. Every paid operation writes a durable `STARTED` event before the call and exactly one terminal event afterward. If a process resumes, it verifies completed bytes and starts only work that is genuinely absent. It cannot silently re-run a paid position and replace its history.

The ledger distinguishes comparison cost from total observed cost. A valid-run comparison may be exact even when a failed attempt has unknown provider cost; conversely, a low candidate score does not make its discarded calls free. Missing cost is therefore marked `partial` or `unavailable`, never zero. Infrastructure-invalid runs remain outside the performance denominator, but their attempts and costs remain visible. A batch with incomplete registered coverage cannot support a formal decision.

3.3 A candidate generator with limited access

An updater should see enough evidence to propose a useful change, but not the evidence that will later judge that change. AGENTLOOPGATE therefore gives it a failure bundle derived from Update-Source and a whitelist of mutable assets in the parent snapshot. Selection and Release tasks, gold answers, evaluator code, private decisions, and the trusted kernel remain unavailable.

Before a proposal becomes a registered candidate, the system materializes it in an isolated tree and checks its path scope, schema, semantic distinctness, runtime capability requirements, and lineage. The updater's native ordering is stored for audit, but it carries no release authority.

3.4 Baseline-paired selection and conditional release

Update-Check is a small, inexpensive screen for malformed or clearly harmful candidates. Candidates that survive it move to Selection, where every candidate and A_0 run on exactly the same tasks. Pairing matters because it reveals whether a candidate's new successes came at the expense of behavior the baseline already performed correctly.

For candidate C, define the paired difference $d_t = y_t(C) - y_t(A_0)$, the gain set $G_C = \{t : d_t = 1\}$, and the regression set $R_C = \{t : d_t = -1\}$. The R15 correctness gate required

$$\sum_t d_t > 0, \qquad R_C = \varnothing, \tag{1}$$

The candidate also needed complete evidence, zero critical violations, no increase in retries or timeouts, a p95 latency ratio no greater than 1.2, and a whole-attempt

cost ratio no greater than 1.2. These conditions are ordered prerequisites, not terms in a weighted score. In particular, lower cost cannot compensate for a correctness regression, and an aggregate-score tie is not an improvement.

When no candidate satisfies every prerequisite, Selection returns HOLD and Release cannot start. If a candidate does pass, Release-ID, Release-OOD, and Replay still require a separately bound authorization. Even SHIP-RECOMMENDED is only a recommendation to a human reviewer; promotion is never automatic.

4 DeepSeek Harness Integration

DeepSeek Harness is a plugin-composed agent runtime whose durable session log records model context, tool calls, and lifecycle events [1]. The integration question is therefore not whether AGENTLOOPGATE should replace that trace. It should not. The host log is already the best account of what the runtime observed. The plugin's job is to connect that account to the evidence chain described above.

AGENTLOOPGATE integrates as an out-of-tree Cordis bundle against a pinned developer-preview revision; the Python core also works without the plugin. The observer listens to public session lifecycle and event interfaces. A live path creates low-latency receipts, and each flush reconciles them with the authoritative `session.events` snapshot. This second path recovers events that may have been missed during construction, resume, or a transient buffer gap. If sequence numbers still contain a hole, the affected evidence is marked incomplete.

The plugin does not register a competing telemetry backend, inspect private SQLite tables, or rewrite native JSONL/SQLite files. A bridge failure is isolated from the agent turn so that governance cannot crash the running agent, but incomplete evidence is not allowed into a formal decision. Unloading the plugin removes its tools, listeners, and child process. In this way developers keep DeepSeek Harness persistence and their existing OTel exporter, while AGENTLOOPGATE adds three reusable capabilities: a verifiable link back to the native trace, normalized run and cost facts, and an evidence-based decision about whether evaluation may advance.

5 System Formation and Failure Experience: P0–R14

AGENTLOOPGATE was not designed in one pass. It emerged from a sequence of frozen experiments in which failures were treated as design evidence. When an experiment exposed a defect, its artifacts were sealed, the defect was diagnosed, and the repair was introduced under a new identity. Results were never rewritten to make an earlier round appear successful. The repository's identities therefore progress from `EXP_BANKING_P0` directly to `EXP_BANKING_R2`; no registered `EXP_BANKING_R1` identity exists. We preserve that gap rather than inventing a run.

These rounds cannot be combined into an effectiveness curve. Protocols, schemas, baselines, and sometimes evaluator inputs changed between them. Their proper role is to explain why the present controls exist. The detailed round-by-round record appears in Table 3; the main development can be understood in three phases.

5.1 Making execution trustworthy

P0–R6 exposed problems close to the model call. A retry budget could reset after process resume, a discarded failed call could disappear from the cost total, and valid tool intent could be rejected because it arrived in an unrecognized wrapper. Repairs introduced a global attempt cap, append-only attempt records, independent Agent and User Simulator ledgers, explicit timeout ownership, and narrow reply normalization. These controls made a crucial distinction possible: infrastructure repair can restore a complete record, but it must not rewrite a genuine task failure into success.

5.2 Making comparison trustworthy

R7–R10 showed that complete executions can still produce an invalid decision. A dynamic price lookup reported zero cost despite nonzero token use; a summary read a different cost plane from the final gate; a broken gold fixture made a correct trajectory appear wrong; and an early Selection design chose among candidates without running the baseline on the same tasks. The successor controls froze pricing, versioned evaluator inputs, propagated one direct cost ledger through every summary, rejected semantically duplicate or unsupported candidates, and made baseline-paired Selection mandatory. This phase also made HOLD a first-class result instead of forcing the best available candidate to win.

5.3 Stopping early without erasing evidence

R11–R14 focused on recovery and fail-fast behavior. The system learned to verify existing bytes rather than regenerate them, isolate updater workspaces, and retain partial batches with an explicit incomplete denominator. R13 then revealed an efficiency defect: after a position became permanently invalid, 12 queued positions still issued 588 model calls. R14 applied position-level fail-fast and prevented 79 authorized positions from starting, while retaining the trigger, its costs, and the partial outcome. The same round exposed one more candidate-validation defect before Selection, leading to validation of materialized candidate bytes in isolated trees.

P0–R14 support a systems claim, not a positive evolution claim. They show a traceable progression from observed failure to executable control. R15 is the first

complete confirmatory use of the baseline-bound policy that resulted.

6 Banking R15 Methodology

R15 asks whether the completed system behaves correctly when an updater produces plausible but uncertain changes. We organize the study around four questions. RQ1 asks whether paired Selection reveals capability trade-offs that an aggregate score hides. RQ2 asks whether the gate stops before Release when no candidate satisfies the frozen policy. RQ3 asks whether attempts, failures, and costs remain complete enough to audit the decision. RQ4 asks whether the DeepSeek Harness plugin can add governance evidence without disrupting native persistence or OpenTelemetry.

6.1 Environment and frozen inputs

The study uses a knowledge-intensive Banking environment derived from the τ family of tool-agent-user benchmarks. These benchmarks test stateful conversation, policy following, API use, and final environment state rather than static text alone [11]. ToolSandbox likewise emphasizes stateful tool-use evaluation [6]. Banking is useful here because a trajectory can look linguistically plausible while producing the wrong state or violating a policy. We use it to exercise the evidence path, not to limit AgentLoopGate to financial applications.

The execution model and user simulator used DeepSeek endpoints with frozen runtime arguments. The external updater was `agentic-harness-engineering` 0.1.0 at commit `8b2a55d97590363fe50c3cc6b5e833b020a4bb4c`. Before formal execution, the benchmark runtime, protocol, study plan, source tree, pricing, and baseline A_0 were all content-addressed.

6.2 Study flow

The study followed the same staged path as the architecture. First, 25 Update-Source runs produced the bounded failure evidence given to the external updater. The updater then generated three semantically distinct candidates, labeled C1–C3 in its native order. Second, Update-Check ran the baseline and all three candidates on 10 tasks each, for 40 positions. Finally, Selection ran the same four variants on an identical set of 15 tasks each, for 60 positions. In total, R15 completed 125/125 authorized pre-release positions across nine batches. Every position was valid, and no Infrastructure Invalid run occurred.

Stable task success was the primary endpoint. The gate also considered latency, critical violations, retries, timeouts, and whole-attempt cost. These were preregistered decision inputs rather than explanations added after seeing the scores. The experiment intentionally contained no Release authorization; Release was not run unless Selection first identified an eligible candidate.

6.3 Paired statistical analysis

For each candidate, the analysis uses the 15 task-level differences d_t. We draw 10,000 deterministic bootstrap resamples and report the nearest-rank 95% interval for the mean paired difference $\bar{d}$. The bootstrap seed was preregistered, and each comparison derives a content-bound effective seed. The task, rather than a model call or retry, is the statistical unit [2].

These intervals describe the frozen Selection population. The three candidates came from one updater and are therefore dependent proposals. We do not make a multiplicity-adjusted superiority claim or infer performance in other domains.

7 Results

7.1 What changed behind the aggregate scores

C1 and C3 matched the baseline's aggregate score of 6/15, while C2 scored 5/15. Looking only at these totals would suggest two ties and one small loss. The paired comparison tells a more important story. Every candidate solved tasks that the baseline missed, but every candidate also broke tasks that the baseline had solved. C1 and C3 each exchanged two gains for two regressions; C2 gained two tasks and regressed three. Table 1 retains the task identifiers so the comparison can be audited, but the release-relevant finding is the shared pattern of capability exchange. None of the paired 95% intervals excluded zero.

Operational results pointed in the same direction. Every candidate exceeded the allowed p95 latency ratio of 1.2, and C1 introduced one additional timeout. Whole-attempt cost ratios stayed below the 1.2 limit but remained higher than the baseline. Because correctness and latency are prerequisites, acceptable cost could not compensate for lost behavior.

7.2 The gate stopped before Release

Since no candidate met every prerequisite, AgentLoopGate returned Hold, selected no governed candidate, and made zero model calls after Selection. No Release-ID, Release-OOD, Replay, promotion, or deployment artifact was created. The machine-readable outcome records the reason as `no_candidate_passed_baseline_bound_selection_policy`.

This result answers RQ1 and RQ2 at the governance level. Paired Selection exposed behavior that aggregate scores concealed, and the conditional boundary stopped the study before additional paid evaluation. It does not

Variant	Stable	Gains	Regress.	Paired $\bar{d}$ [95% CI]	p95 ratio	Cost ratio	Disposition
A_0	6/15	–	–	–	1.000	1.000	baseline
C1	6/15	017, 096	062, 095	0.000 [−.267, .267]	1.882	1.121	held
C2	5/15	017, 090	056, 062, 095	−.067 [−.333, .200]	1.670	1.108	held
C3	6/15	017, 090	062, 095	0.000 [−.267, .267]	1.652	1.127	held

Table 1: Results on the frozen 15-task Selection set. "Gains" and "Regress." list audit identifiers for tasks that changed relative to A_0. Cost includes all observed attempts, and p95 latency is normalized to A_0. No candidate satisfied the correctness and latency requirements.

Figure 2: Left: each candidate gained and lost tasks relative to A_0. Right: all candidates failed correctness and latency prerequisites, so the gate returned HOLD. The panels summarize Selection only; Release was not run. Both are deterministic views of the same content-bound result.

say that the candidate generator is incapable of improvement; it says that this candidate set did not provide the evidence required for release.

7.3 The decision has a complete cost and attempt record

RQ3 concerns whether the outcome can be audited, including work that did not contribute to a success score. All nine batches reconciled exactly. Formal evaluation cost USD 3.8830976464, and the external updater cost USD 0.0255671416, for total known model cost of USD 3.9086647880. There was no unknown model-cost scope. Local compute and GitHub Actions cost were marked unmetered or unavailable rather than incorrectly reported as zero.

The supporting ledger contains 4,378 terminal model calls, 10,343,844 input tokens, 4,648,289 output tokens, and 222,019,328 cache-read tokens. It contains no unresolved calls and no provider-level retries; task-level retries and timeouts remain separately visible in batch evidence. The formal process took 52,591.59 seconds (14.61 hours), in part because it used a deliberately serial, low-concurrency path. This is not a lower bound on runtime. It is a record of the actual path whose evidence supports the decision.

7.4 The plugin preserved host behavior in fixtures

RQ4 was evaluated with a no-model, headless fixture rather than the paid Banking runs. For both JSONL and SQLite persistence, the fixture ran 30 paired iterations with 100 logical session events per iteration. Native persistence survived every run, the session-event hashes matched, observer coverage was complete, and OTel coexistence remained enabled.

Measured overhead was small but noisy: JSONL had p50 −0.308 ms and p95 16.424 ms, while SQLite had p50 0.244 ms and p95 0.542 ms. Negative samples are ordinary local timing noise. These results show lifecycle coexistence in the fixture; they are not production throughput or tail-latency claims.

A separate synthetic control checked the integrity gate itself. Disabling the integrity prerequisite would make the fixture return SHIP-RECOMMENDED, whereas the production gate returns HOLD on `evaluation_integrity`. This control demonstrates software behavior, not a causal estimate from R15.

8 Discussion

8.1 Candidate generation is not yet positive evolution

The updater produced three executable harness changes, and all three altered the task-success pattern. This shows that the proposal mechanism was active; it does not show

that the agent improved. Positive evolution would require a candidate to add stable behavior while preserving protected baseline behavior and satisfying the operational limits. None did so.

The paired failures nevertheless suggest a future research direction. An updater could treat the baseline-success set as explicit constraints and make smaller changes aimed at one failure cluster at a time. That idea is a hypothesis derived from R15, not a result of R15. Testing it would require a new frozen identity and untouched confirmation tasks.

8.2 Why the gate is ordered rather than weighted

A single weighted score is attractive because it always produces a ranking. It also allows one property to compensate for another. A candidate might receive credit for lower cost even after breaking policy-important behavior, or for a new success that replaces an old one. In a release setting, those trades should be explicit decisions, not consequences hidden inside a coefficient.

AgentLoopGate therefore evaluates prerequisites in order: evidence must first be complete, protected correctness must then be preserved, and only then do latency and cost determine operational non-inferiority. Applications may adopt different frozen limits, but changing a limit after seeing results creates a new study rather than a reinterpretation of the old one.

8.3 Why native traces remain authoritative

Observability and evaluation evidence overlap, but they answer different questions. A native session log records what the runtime observed. An OTel exporter records what configured instrumentation sent elsewhere. A governance receipt establishes whether the first record was complete, bound to the right experiment, and included in the decision. Keeping these roles distinct lets developers retain existing persistence and telemetry while adding stronger release semantics.

8.4 Abstention is the primary systems result

Research on agent improvement naturally emphasizes examples where a method raises an aggregate score. A release-governance system must also work when no proposal deserves to advance. R15 exercises that branch: the candidates were plausible and executable, but their improvements were coupled to regressions. The main result is therefore not a better Banking agent. It is a complete, costed explanation of why the system stopped, together with evidence that no Release tail began.

9 Related Work

9.1 Improving language agents

ReAct interleaves reasoning and acting [12]; Reflexion stores linguistic feedback across trials [10]; and Self-Refine iteratively critiques outputs [7]. DSPy compiles declarative LM pipelines against metrics [4]. AgentOptimizer treats functions as learnable agent parameters [13], while Automated Design of Agentic Systems searches over agent programs [3]. These methods address how to generate or optimize behavior. AgentLoopGate addresses the subsequent release question: it constrains the evidence available to a generator, authenticates outcomes, evaluates proposals independently, and decides whether evaluation may advance.

9.2 Evaluating tool-using agents

AgentBench evaluates agents across interactive environments [5]; τ-bench focuses on tool-agent-user interaction and reliability [11]; ToolSandbox adds stateful on-policy tool execution [6]; and ToolEmu uses an emulated sandbox to identify risky tool behavior [9]. Such environments produce rich execution evidence. AgentLoopGate adds the experiment lifecycle around that evidence: split isolation, candidate lineage, attempt and cost accounting, and a conditional boundary between Selection and Release.

9.3 Reproducible statistical decisions

Classical bootstrap methods quantify uncertainty from empirical populations [2]. AgentLoopGate binds the exact population, seed, batch digests, and comparison artifact so that an interval cannot drift independently of the runs it summarizes. The contribution is operational rather than a new bootstrap method: statistical output becomes one checkable component of a larger decision path, not a substitute for evidence integrity.

10 Limitations and Ethics

R15 is a focused systems validation, not a broad effectiveness study. It covers one Banking environment, one updater backend, one model/runtime family, three dependent candidates, and 15 Selection tasks. The bootstrap intervals are therefore wide and descriptive. Since no candidate reached Release, the paper contains no Release-ID, OOD, Replay, or cross-domain effectiveness evidence. The plugin overhead study is a local synthetic fixture on arm64 macOS rather than a production load test. Provider variance, user simulation, and the lack of a formal human-factors study further limit generalization.

The study publishes sanitized aggregates rather than raw customer or session content. Raw traces, prompts, model responses, credentials, and hidden benchmark

data remain private. Clean-room verification scans intended public artifacts for secrets and direct PII. Human-controlled promotion reduces the risk of an accidental release, but these controls do not establish safety against a malicious host, a compromised evaluator, or every form of prompt injection.

11 Reproducibility and Artifact Availability

The repository contains the Python governance core, the DeepSeek Harness bundle, a no-key demonstration, frozen configuration identities, unit and integration tests, and a sanitized 18-file R15 v1.1 result package. The package can be verified without private traces. Its Manifest digest is `sha256:79e9a8dd31009f969cdd79021bbcc857c827dc1d5a6808a28fd05937e4f364c8`; the private Outcome digest it binds is `sha256:827ed2a5b3337832c49cd255f17568fad3a58fe201860311d66decb83ba0bdc3`.

One clean-room command runs the Python tests and demonstration, builds the source and wheel distributions, type-checks and tests the plugin, executes the headless conformance fixture, packs the bundle, verifies both R15 packages, and scans the intended public tree for secrets and direct PII. The repository is prepared at the URL in the title block. Repository visibility and arXiv submission remain separate owner-authorized actions at the time of this draft.

12 Conclusion

Agent harnesses can generate useful changes, but generation alone cannot decide what should be released. AgentLoopGate separates those responsibilities. It preserves the host runtime's own account of execution, binds that account to immutable study identities, records failed as well as successful attempts, and compares each candidate with the baseline on the same tasks.

In Banking R15, every candidate added some behavior and every candidate also removed protected behavior. Aggregate scores hid part of that exchange; paired Selection exposed it. The appropriate result was Hold with zero Release calls. This is a negative result for the candidate set but a positive result for governance: a self-evolution claim must be supported by complete evidence, and the system must be able to explain when that evidence is not enough.

References

[1] DeepSeek AI. Deepseek harness: Everything is a plugin. `https://github.com/deepseek-ai/deepseek-harness`, 2026. Developer preview; accessed 2026-08-25.

[2] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[3] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

[4] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

[5] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating LLMs as agents. *International Conference on Learning Representations*, 2024.

[6] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. *arXiv preprint arXiv:2408.04682*, 2024.

[7] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.

[8] OpenTelemetry Authors. Opentelemetry specification. `https://opentelemetry.io/docs/specs/otel/`, 2026. Accessed 2026-08-25.

[9] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. *International Conference on Learning Representations*, 2024.

[10] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

[11] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.

[13] Shaokun Zhang, Jieyu Zhang, Jiale Liu, Linxin Song, Chi Wang, Ranjay Krishna, and Qingyun Wu. Offline training of language model agents with functions as learnable weights. *arXiv preprint arXiv:2402.11359*, 2024.

A Evidence Binding

Table 2 lists the principal content identities used by this paper. Full values are included so that the manuscript can be checked against the sanitized package without private execution data.

Artifact	Digest or identity
Experiment	`EXP_BANKING_R15`
Protocol	`sha256:b40198176f9de2b39c9433131455bc0af07eee54650327622677cd9085b84a5f`
Study	`sha256:bd5988f4f89068fa6347728f6b88ee4b8d8d1f644931ec110944e06b74e25043`
Execution source	`tree:sha256:0e3d0794bd21c5f20ff78512144e50da37b16a9784fa948a13055f6c847b4c8e`
Selection	`sha256:b5e800793f700e1d096a325961b3f40bac8cd93d12dddbdcbc111d48fa600466`
Statistics	`sha256:8bc2da84cefc7ed100947401a6277d9fab49109ce29d065f91b064a3a2f5b8ba`
Outcome	`sha256:827ed2a5b3337832c49cd255f17568fad3a58fe201860311d66decb83ba0bdc3`
Public package	`sha256:79e9a8dd31009f969cdd79021bbcc857c827dc1d5a6808a28fd05937e4f364c8`

Table 2: Content-addressed identities for the R15 result.

B System-Formation Record

Table 3 preserves the round-level counts, costs, failures, and successor controls summarized in Section 5. These values are historical observations, not comparable measurements of candidate effectiveness.

Table 3: Detailed system-formation evidence for P0–R14. Every successor kept the predecessor's raw evidence and used a new frozen identity whenever execution semantics changed.

Round	Terminal evidence	Failure exposed	Control carried forward
P0–R2	P0 preserved as superseded HOLD; R2 A4 retained a failed protocol diagnosis	Frozen one-retry intent differed from the invoked runtime default; stale acceptance, source packaging, ignore rules, and Linux async behavior also failed clean-room checks	Content-address the execution protocol; supersede rather than rewrite; require source-artifact and clean-checkout Linux verification before paid evidence
R3	22/25 valid Update-Source positions; HOLD	Outer resume reapplied the per-process retry budget, and discarded attempts lost User Simulator cost	Global per-position attempt cap; append-only task-attempt and independent Agent/User usage ledgers; unknown cost is never zero
R4	24/25 valid; HOLD; exact whole-attempt model cost USD 0.6917880208	A recurrent, uniquely mappable reply wrapper remained unrecognized	Narrow allow-list normalization only when tool identity and arguments are explicit; ambiguous syntax still fails closed
R5–R6	R5: 23/25 valid; R6: 24/25 valid; both HOLD. R6 reconciled exact whole-attempt cost USD 0.7698343648	Reasoning-only empty finals, equal inner and outer timeouts, additional explicit reply shapes, and indirect failed-call lineage	One separately ledgered same-session final-only repair; ordered timeouts; direct task/attempt/session joins; no inference from hidden reasoning
R7–R8	R7 completed Source, created three candidates, and completed A0 Check; the first candidate Check failed partial. R8 made zero paid calls	Dynamic pricing produced false exact-zero User cost for nonzero tokens; later audit found that summaries and gates still read raw rather than direct-ledger cost	Frozen-price recomputation with a positive-token invariant; direct cost must propagate through summaries, curves, and final gates end to end
R9	25/25 Source valid; immutable HOLD; exact whole-attempt cost USD 0.7015112488	Task 053's golden fixture omitted a required user action, making a correct trajectory look wrong	Deterministic no-model cause isolation; versioned evaluator overlay; block candidate generation under contaminated evidence
R10	95 formal positions; paid HOLD; exact model cost through C2 USD 3.0651904832	Selection omitted A_0, forced a winner, ignored several whole-attempt operational facts, admitted duplicate candidates, and allowed an unbound tool route	Same-pool baseline pairing; first-class abstention; whole-attempt retry/timeout/p95/cost inputs; semantic deduplication and runtime capability binding

Round	Terminal evidence	Failure exposed	Control carried forward
R11–R12	R11: 24/25 Source valid. R12: 34/35 executed positions valid; both HOLD. R12 observed Provider-cost lower bound USD 1.1970712488	Failed-attempt sealing lacked direct lineage; a User Simulator empty message and updater use of process-global `/tmp` broke successor execution	Verify-only recovery for existing bytes; bounded final-only User repair; attempt-local updater runtime and sandbox doctor; every repair requires a new identity
R13	24/25 Source valid; HOLD. After permanent invalidity, 12 queued positions still made 588 calls costing a known USD 0.4686327800	Cross-stage gating worked, but the serial batch lacked position-level fail-fast	Stop before the next position after retry exhaustion while retaining the failed position, unknown cost scope, native trace, and incomplete denominator
R14	46/125 positions executed; HOLD; exact cost USD 1.4876297096. Fail-fast prevented 79 authorized positions from starting	Candidate precheck inspected baseline bytes and patch tokens rather than validating materialized prospective bytes; the only complete candidate Check regressed a baseline success	Apply each patch in an isolated tree and validate final bytes before registration; seal a first-class partial HOLD; resume is verification-only

Table 4 maps the narrative to tracked historical records. These artifacts remain context for the system design; they are not part of the R15 effectiveness population.

Rounds	Principal repository evidence
P0–R4	`configs/formal_experiment.yaml`; `docs/adr/0002-version-formal-execution-protocol-for-banking-r2.md`; `SPEC.md` Sections 16.4–16.6; R3/R4 execution and reply calibrations under `artifacts/research/`
R5–R8	`artifacts/research/banking_r5/execution_diagnosis.json`; `banking_r6/execution_diagnosis.json`; `banking_r7/execution_diagnosis.json`; `banking_r8/cost_incident_diagnosis.json`
R9–R10	`artifacts/research/banking_r9/experiment_hold.json`; `banking_r9/eval_incident_task_053.json`; `banking_r10/selection_design_correction.json`
R11–R12	`docs/research/banking-r11-preregistration.md`; `artifacts/research/banking_r12/formal_execution_seal.json`; `docs/research/banking-r12-successor-plan.md`
R13–R14	`artifacts/research/banking_r13/formal_execution_seal.json`; `banking_r13/fail_fast_incident.json`; `banking_r14/formal_execution_seal.json`; `docs/research/banking-r14-terminal.md`

Table 4: Audit sources for the system-formation narrative. Artifact paths after the first full prefix remain relative to `artifacts/research`.

C Candidate-Level Operational Values

The baseline whole-attempt cost was USD 0.4520428360. C1, C2, and C3 cost USD 0.5066522160, USD 0.5009778704, and USD 0.5095613824. Their p50/p95 latencies in milliseconds were 264,917/1,838,752; 273,388/1,631,601; and 314,154/1,614,647, compared with A_0 at 265,903/977,157. C1 and A_0 each recorded one task retry; only C1 recorded a timeout. No candidate had a critical violation or an Infrastructure Invalid run.

D Claim-to-Artifact Audit Map

Table 5: Claim-level map within the 18-file sanitized R15 v1.1 package. Paths are relative to `artifacts/research/banking_r15/release_v2`.

Paper claim	Sanitized R15 evidence source
Selection scores, paired gain/regression sets, gate findings, and governed null candidate	`selection.json`; `reports/selection_hold.json`
Paired bootstrap intervals, statistical unit, seeds, and multiplicity scope	`statistics.json`
Exact batch and whole-experiment model cost	`cost_summary.json`
Calls, tokens, provider retries, failures, and operation duration	`failure_accounting.json`
Zero post-Selection calls, zero Release batches, and terminal HOLD	`selection_hold_outcome.json`
Plugin persistence/OTel coexistence and synthetic integrity control	`ablations/plugin_coexistence_overhead.json`; `ablations/integrity_gate.json`
Frozen source, protocol, study, and package identities	`reproduction.json`; `manifest.json`

The repository includes a non-model verifier that checks the manuscript's author identity, P0–R14 formation claims, R15 decision, scores, paired changes, intervals, costs, usage, ablations, digests, citations, claim limits, and figure presence against tracked artifacts:

```
uv run python -m scripts.verify_arxiv_paper --project .
```

Rebuilding or checking this manuscript makes no provider model calls and does not re-run R15. Formal experiment replay would require a new frozen identity and remains outside the paper build.